



Manual para Iniciar un Nuevo Proyecto

➤ Arquitectura de 3 Capas: Directives → Orchestration → Execution

Autor: Agente IA (Orquestador)

Tipo: Guía de inicio

Fecha: 13 de agosto de 2026

Índice

1. Introducción	2
2. Estructura de Directorios	2
3. Paso 1 — Archivos del Agente	2
4. Paso 2 — Configuración del Agente	3
5. Paso 3 — Primeras Directivas	3
6. Paso 4 — Scripts de Ejecución	4
7. Paso 5 — Archivos de Configuración	5
7.1. .gitignore	5
7.2. .env	5
8. Flujo de Trabajo del Agente	6
9. Buenas Prácticas	6
10. Conclusión	7

1 Introducción

Este manual describe los pasos para iniciar un nuevo espacio de trabajo bajo la arquitectura de 3 capas (Directives → Orchestration → Execution). El objetivo es proporcionar una estructura reutilizable que maximice la fiabilidad, separando la lógica probabilística del LLM de la ejecución determinista mediante scripts especializados.

Arquitectura en 3 capas

 Directives	<i>Qué hacer</i> — SOPs en YAML (Layer 1).
 Orquestación	<i>Decide cuándo y cuál</i> — validación y enrutamiento (Layer 2).
 Ejecución	<i>Cómo</i> — scripts deterministas (Layer 3).

2 Estructura de Directorios

Crear la siguiente estructura en la raíz del nuevo proyecto:

Árbol de directorios

```
1 proyecto/
2 |-- .agent/           # Instrucciones del sistema para el agente IA
3 |-- directives/       # SOPs en YAML (Layer 1)
4 |-- execution/        # Scripts deterministas (Layer 3)
5 |-- .tmp/            # Archivos temporales (regenerables)
6 |-- .env              # Credenciales y configuracion (no committear)
7 |-- opencode.json     # Configuracion del agente
8 |-- .gitignore        # Exclusiones de seguimiento
9 |-- requirements.txt   # Dependencias (se completa al definir el
                        proyecto)
```

3 Paso 1: Archivos del Agente (.agent/)

Copiar los archivos `FRAMEWORK.md` e `INSTRUCTIONS.md` desde la plantilla ubicada en `docs/AGENTE_IA/` a la carpeta `.agent/` del nuevo proyecto. Estos archivos definen el marco operativo y las instrucciones del agente.

Personalizar en `INSTRUCTIONS.md`

- **Sección 1:** Ajustar el tono, las restricciones de entorno y el stack tecnológico (SO, gestor de paquetes, lenguajes).
- **Sección 8:** Definir el formato de documentación del proyecto (Markdown, LaTeX, etc.).
- **Sección 10:** Completar las directrices de dominio técnico: lenguaje principal, frameworks, infraestructura, seguridad y flujo de trabajo.

Actualizar en `FRAMEWORK.md`

- **Available Directives:** Listar los flujos de trabajo repetibles propios del proyecto.
- **Setup:** Documentar los pasos de instalación específicos del proyecto.

4 Paso 2: Configuración del Agente (`opencode.json`)

Crear `opencode.json` en la raíz del proyecto para que el agente cargue las instrucciones desde `.agent/`:

`opencode.json`

```
1 {  
2   "$schema": "https://opencode.ai/config.json",  
3   "instructions": [".agent/*.md"]  
4 }
```

5 Paso 3: Crear las Primeras Directivas (`directives/`)

Cada directiva es un archivo YAML que describe un flujo de trabajo repetible. Estructura recomendada:

`directives/mi_flujo.yaml`

```
1 # directives/mi_flujo.yaml
2 goal: "Descripcion del objetivo"
3 required_inputs:
4   - name: "entrada_1"
5     description: "Que datos se necesitan"
6 steps:
7   - script: "execution/mi_script.py"
8     mapping:
9       entrada_1: "{entrada_1}"
10 expected_outputs: "Descripcion del resultado esperado"
11 edge_cases:
12   - "Que hacer si algo falla"
```

Crear al menos una directiva inicial para validar que el flujo funciona. Ejemplos comunes: `scrape_website.yaml`, `research_topic.yaml`, `git_update.yaml`.

6 > Paso 4: Crear los Scripts de Ejecución (`execution/`)

Cada script debe cumplir:

Requisitos de un script

- Una sola responsabilidad.
- Entradas por argumentos CLI.
- Secretos en `.env`.
- Salida por stdout (preferiblemente JSON).
- Códigos de salida: 0 (éxito), 1+ (fallos categorizados).
- Validar sus propias salidas y fallar ruidosamente si algo está mal.

Ejemplo mínimo en Python:

Plantilla de script

```
1  #!/usr/bin/env python3
2  """Descripcion breve de la funcion del script."""
3  import sys
4
5  def main():
6      # Validar entradas
7      # Ejecutar logica
8      # Validar salida
9      print('{ "status": "ok" }')
10     return 0
11
12 if __name__ == "__main__":
13     sys.exit(main())
```

7 Paso 5: Archivos de Configuración

7.1 .gitignore

Crear un .gitignore genérico:

.gitignore

```
1  __pycache__/  
2  *.pyc  
3  .env  
4  .tmp/  
5  chroma_db/  
6  *.aux  
7  *.log  
8  *.out  
9  *.toc  
10 venv/  
11 .venv/
```

7.2 .env

Plantilla para credenciales:

`.env`

```
1 # API Keys
2 # API_KEY=tu_clave_aqui
```

8 Flujo de Trabajo del Agente

Una vez configurada la estructura, el agente opera siguiendo este ciclo:

Ciclo de operación

1. **Validación de Entorno:** Verificar que el sistema cumple los requisitos.
2. **Búsqueda:** Revisar `directives/` para ver si existe un SOP para la tarea solicitada.
3. **Planificación:** Seleccionar la directiva y el script de ejecución correspondiente.
4. **Ejecución:** Invocar el script con las entradas validadas.
5. **Validación:** Confirmar que la salida coincide con lo esperado.
6. **Registro:** Guardar el estado en `.tmp/run_state.json`.
7. **Notificación:** Informar al usuario del resultado.
8. **Aprendizaje:** Si surge un error, documentarlo y actualizar la directiva.

9 Buenas Prácticas

Recomendaciones

- **Reutilizar antes de construir:** Revisar `execution/` antes de crear un nuevo script.
- **Directivas atómicas:** Cada directiva debe describir un solo flujo de trabajo.
- **Versionado de directivas:** Si cambia la lógica, conservar versiones antiguas (v1, v2).
- **Máximo 3 reintentos:** Si un flujo falla 3 veces, escalar al usuario.
- **Validar siempre:** Después de cada paso, confirmar esquema, cantidades y archivos generados.
- **Documentar errores:** Mantener un registro de errores comunes y sus soluciones (ej. `errores.md`) para evitar regresiones.

10 Conclusión

Resultado

Con esta estructura, el nuevo proyecto queda listo para operar bajo la arquitectura de 3 capas. El esqueleto inicial es mínimo pero escalable: las directivas, scripts y configuraciones se agregan según el proyecto va tomando forma.